LaRS Dataset Embedding Analysis

Dataset Overview

The **LaRS** (Lake/River/Sea) dataset is used in the MaCVi 2026 embedded semantic-segmentation challenge track. It covers maritime scenes with three semantic classes water, obstacles, and sky.

Split	Images	Notes
train	2,605	Labelled images
val	198	Labelled images
test	1,203	Labels withheld
total	**4,006**	

Typical resolutions: 1278 x 958 (landscape) and 1280 x 720 (widescreen HD).

Model Choice: NVIDIA C-RADIOv3-H

RADIO (Radio Amplitude and Distortion Importance Optimization) is a vision foundat model trained by NVIDIA as a multi-teacher distillation of DINOv2, SAM, CLIP, and otl experts. RADIO-H is the largest variant (ViT-H backbone, ~632 M parameters).

Reasons for choosing RADIO over other encoders:

- **Spatial richness**: RADIO retains full spatial token grids, not just a [CLS] token, g per-region sensitivity critical for detecting small maritime obstacles.
- **Resolution flexibility**: `get_nearest_supported_resolution()` snaps images valid resolution, avoiding padding artifacts with non-standard aspect ratios.
- **No explicit crop extraction**: Instead of manually cropping regions of interest, the model attend to the whole scene. The spatial token average is a compact, semantically rich descriptor that is more robust to unknown object positions than based methods.
- **Strong OOD sensitivity**: RADIO was trained on diverse web-scale data, so its embedding space reliably reflects visual distribution shifts.

Resolution Choice

Images are processed at full original resolution, snapped to the nearest resolution su by RADIO via `get_nearest_supported_resolution()`. This avoids quality loss from downscaling while keeping inference stable.

What Each Section Shows

Section	Content	Why
2	Embedding extraction	Slow; runs once, writes to SQLite
3	Load embeddings	Fast; describe split/sequence statistics
4	t-SNE computation	Medium; runs once, saved to .npy
5	t-SNE by split	Check if train/val/test overlap
6	t-SNE by sequence	Detect sequence-level clusters or outliers
7	Cosine similarity (OOD)	Quantify how well training data covers test
8	Report generation	Export to PDF
9	Summary table	One-page overview

Interpretation Guide

- **t-SNE clusters**: Tight clusters by split indicate good coverage; isolated test poir suggest scenes not represented in training.
- **Max cosine similarity**: For each test image we find the most similar training im Low max-sim (< 0.70) indicates potential OOD samples that may be harder for a
- **Sequence colouring**: If a sequence forms a tight cluster far from training data, consider adding augmentation or more training data for that environment.

Section 1 - Setup & Configuration

In [ ]:
```
import os
import sys
import sqlite3
import base64
import io
import time
from pathlib import Path

import numpy as np
import pandas as pd
from tqdm.auto import tqdm
from PIL import Image

import torch
import plotly.express as px
import plotly.graph_objects as go

# ---- Paths ----
ANALYSIS_DIR = Path('/home/farms/macvi2026-farms-team/challenges/common/l
DATASET_ROOT = Path('/home/farms/data/lars_dataset/img_dir')
DB_PATH      = ANALYSIS_DIR / 'embeddings.db'
TSNE_PATH    = ANALYSIS_DIR / 'tsne_results.npy'
REPORT_PATH  = ANALYSIS_DIR / 'report.pdf'
LOG_PATH     = ANALYSIS_DIR / 'logs' / 'analysis.log'
```

```
# ---- Model config ----
MODEL_NAME_H = 'nvidia/C-RADIOv3-H'
MODEL_NAME_L = 'nvidia/C-RADIOv3-L'
BATCH_SIZE   = 1
COMMIT_EVERY = 50

# ---- VRAM check ----
print('PyTorch version :', torch.__version__)
print('CUDA available  :', torch.cuda.is_available())
if torch.cuda.is_available():
    for i in range(torch.cuda.device_count()):
        props = torch.cuda.get_device_properties(i)
        total_gb = props.total_memory / 1024**3
        alloc_gb = torch.cuda.memory_allocated(i) / 1024**3
        print(f'  GPU {i}: {props.name}  total={total_gb:.1f} GB  alloc={

DEVICE = 'cuda' if torch.cuda.is_available() else 'cpu'
print('Using device:', DEVICE)
```

Section 2 - Embedding Extraction

Slow cell -- skipped if `embeddings.db` already contains all 4006 rows.

In [ ]:
```
# ---- helpers ----

def get_image_list(split):
    list_file = DATASET_ROOT / split / 'image_list.txt'
    names = list_file.read_text().strip().splitlines()
    records = []
    for name in names:
        parts = name.rsplit('_', 1)
        sequence = parts[0] if len(parts) > 1 else name
        img_path = DATASET_ROOT / split / 'images' / (name + '.jpg')
        records.append((name, split, sequence, img_path))
    return records


def init_db(conn):
    conn.executescript("""
        CREATE TABLE IF NOT EXISTS metadata (
            key   TEXT PRIMARY KEY,
            value TEXT
        );
        CREATE TABLE IF NOT EXISTS embeddings (
            id                    INTEGER PRIMARY KEY AUTOINCREMENT,
            img_name              TEXT    NOT NULL,
            split                 TEXT    NOT NULL,
            sequence              TEXT    NOT NULL,
            width                 INTEGER,
            height                INTEGER,
            summary_embedding     BLOB,
            spatial_mean_embedding BLOB
        );
        CREATE INDEX IF NOT EXISTS idx_split    ON embeddings(split);
        CREATE INDEX IF NOT EXISTS idx_sequence ON embeddings(sequence);
    """)
    conn.commit()
```

```
def get_already_done(conn):
    return {r[0] for r in conn.execute('SELECT img_name FROM embeddings')


def blob(arr):
    return arr.astype(np.float32).tobytes()


TOTAL_IMAGES = 4006


def count_done():
    if not DB_PATH.exists():
        return 0
    with sqlite3.connect(DB_PATH) as c:
        return c.execute('SELECT COUNT(*) FROM embeddings').fetchone()[0]


if count_done() >= TOTAL_IMAGES:
    print('embeddings.db already has', count_done(), 'rows -- skipping ex
else:
    print('Starting extraction (', count_done(), '/', TOTAL_IMAGES, 'alre

    from transformers import AutoModel
    import gc

    model_name = MODEL_NAME_H
    try:
        print('Loading', model_name, '...')
        radio_model = AutoModel.from_pretrained(model_name, trust_remote_
        radio_model = radio_model.to(DEVICE).eval()
        with torch.no_grad(), torch.autocast(DEVICE, dtype=torch.bfloat16
            probe = torch.zeros(1, 3, 512, 512, device=DEVICE)
            _ = radio_model(probe)
        del probe
        torch.cuda.empty_cache()
        print('Loaded', model_name, 'successfully.')
    except torch.cuda.OutOfMemoryError:
        print('OOM with', model_name, ', falling back to', MODEL_NAME_L)
        model_name = MODEL_NAME_L
        del radio_model
        gc.collect(); torch.cuda.empty_cache()
        radio_model = AutoModel.from_pretrained(model_name, trust_remote_
        radio_model = radio_model.to(DEVICE).eval()
        print('Loaded', model_name, 'successfully (fallback).')

    ACTUAL_MODEL = model_name
    print('Model used:', ACTUAL_MODEL)

    all_records = []
    for split in ('train', 'val', 'test'):
        all_records.extend(get_image_list(split))
    print('Total records:', len(all_records))

    conn = sqlite3.connect(DB_PATH)
    init_db(conn)
    already_done = get_already_done(conn)
```

```
    conn.execute("INSERT OR IGNORE INTO metadata VALUES (?,?)", ('model_n
    conn.execute("INSERT OR IGNORE INTO metadata VALUES (?,?)", ('created
    conn.commit()

    pending = [(n, sp, seq, p) for n, sp, seq, p in all_records if n not
    print('Pending:', len(pending), 'images')

    embed_dim = None
    batch_data = []

    def flush_batch(data, conn):
        conn.executemany(
            'INSERT INTO embeddings '
            '(img_name, split, sequence, width, height, summary_embedding
            'VALUES (?,?,?,?,?,?,?)',
            data,
        )
        conn.commit()


    with tqdm(total=len(pending), desc='Extracting embeddings') as pbar:
        for i, (img_name, split, sequence, img_path) in enumerate(pending
            try:
                img_pil = Image.open(img_path).convert('RGB')
                W, H = img_pil.size
                th, tw = radio_model.get_nearest_supported_resolution(H,
                img_resized = img_pil.resize((tw, th), Image.BICUBIC)
                img_t = torch.from_numpy(
                    np.array(img_resized).astype(np.float32) / 255.0
                ).permute(2, 0, 1).unsqueeze(0).to(DEVICE)

                with torch.no_grad(), torch.autocast(DEVICE, dtype=torch.
                    output = radio_model(img_t)

                if isinstance(output, (list, tuple)) and len(output) >= 2
                    summary_t, spatial_t = output[0], output[1]
                elif hasattr(output, 'summary') and hasattr(output, 'spat
                    summary_t  = output.summary
                    spatial_t  = output.spatial_features
                else:
                    spatial_t = output
                    summary_t = spatial_t.mean(dim=1) if spatial_t.ndim =

                summary_vec = summary_t.squeeze(0).float().cpu().numpy()
                if spatial_t.ndim == 3:
                    spatial_mean = spatial_t.squeeze(0).float().cpu().num
                else:
                    spatial_mean = spatial_t.squeeze(0).float().cpu().num

                if embed_dim is None:
                    embed_dim = spatial_mean.shape[0]
                    conn.execute("INSERT OR IGNORE INTO metadata VALUES (
                    conn.commit()

                batch_data.append((img_name, split, sequence, W, H, blob(

                if len(batch_data) >= COMMIT_EVERY:
                    flush_batch(batch_data, conn)
                    batch_data = []
```

```
            except Exception as exc:
                print('ERROR on', img_name, ':', exc)

            pbar.update(1)
            pbar.set_postfix({'processed': str(i+1) + '/' + str(len(pendi

    if batch_data:
        flush_batch(batch_data, conn)
    conn.close()

    with sqlite3.connect(DB_PATH) as c:
        if embed_dim is not None:
            c.execute("INSERT OR REPLACE INTO metadata VALUES (?,?)", ('e
            c.execute("INSERT OR REPLACE INTO metadata VALUES (?,?)", ('m
        c.commit()

    print('Extraction complete:', count_done(), '/', TOTAL_IMAGES)
```

Section 3 - Load Embeddings

In [ ]:

```
import sqlite3
import numpy as np
import pandas as pd


def load_embeddings(db_path):
    con = sqlite3.connect(db_path)
    rows = con.execute(
        'SELECT id, img_name, split, sequence, width, height, '
        'summary_embedding, spatial_mean_embedding FROM embeddings ORDER
    ).fetchall()
    con.close()
    records, summary_list, spatial_list = [], [], []
    for row in rows:
        _id, img_name, split, seq, w, h, sum_blob, spa_blob = row
        records.append({'id': _id, 'img_name': img_name, 'split': split,
                        'sequence': seq, 'width': w, 'height': h})
        summary_list.append(np.frombuffer(sum_blob, dtype=np.float32).cop
        spatial_list.append(np.frombuffer(spa_blob, dtype=np.float32).cop
    df = pd.DataFrame(records)
    return df, np.stack(summary_list), np.stack(spatial_list)


def load_metadata(db_path):
    con = sqlite3.connect(db_path)
    meta = dict(con.execute('SELECT key, value FROM metadata').fetchall()
    con.close()
    return meta


meta = load_metadata(DB_PATH)
print('Database metadata:')
for k, v in meta.items():
    print('  ', k, ':', v)

df, summary_arr, spatial_arr = load_embeddings(DB_PATH)
print('Loaded', len(df), 'embeddings, spatial_dim=', spatial_arr.shape[1]
```

```
print('=== Count per split ===')
print(df.groupby('split').size().reset_index(name='n_images').to_string(i

print('=== Sequences per split ===')
print(df.groupby('split')['sequence'].nunique().reset_index(name='n_seque

print('=== Top-10 sequences by image count ===')
print(df.groupby('sequence').size().sort_values(ascending=False).head(10)
```

Section 4 - t-SNE

Medium cell -- skipped if `tsne_results.npy` already exists.

In [ ]:
```
from pathlib import Path

if TSNE_PATH.exists():
    tsne_result = np.load(TSNE_PATH)
    print('Loaded t-SNE from', TSNE_PATH, 'shape=', tsne_result.shape)
else:
    print('Computing t-SNE ...')
    X = spatial_arr

    from sklearn.decomposition import PCA
    n_pca = min(50, X.shape[1], X.shape[0] - 1)
    print('PCA:', X.shape[1], '->', n_pca, 'dims')
    pca = PCA(n_components=n_pca, random_state=42)
    X_pca = pca.fit_transform(X)
    print('PCA explained variance:', round(pca.explained_variance_ratio_.

    tsne_backend = 'unknown'
    try:
        from cuml.manifold import TSNE as cumlTSNE
        import cudf
        print('Using cuML t-SNE (GPU)')
        tsne_backend = 'cuml'
        tsne = cumlTSNE(n_components=2, perplexity=30, n_iter=1000, rando
        X_pca_gdf = cudf.DataFrame(X_pca.astype(np.float32))
        tsne_result = tsne.fit_transform(X_pca_gdf).to_numpy()
    except Exception as e:
        print('cuML t-SNE failed (', e, '); falling back to sklearn')
        tsne_backend = 'sklearn'
        from sklearn.manifold import TSNE
        tsne = TSNE(n_components=2, perplexity=30, n_iter=1000, random_st
        tsne_result = tsne.fit_transform(X_pca)

    np.save(TSNE_PATH, tsne_result)
    print('t-SNE done (backend=', tsne_backend, '), saved to', TSNE_PATH)

df['tsne_0'] = tsne_result[:, 0]
df['tsne_1'] = tsne_result[:, 1]
print('t-SNE coordinates added to DataFrame.')
```

Section 5 - t-SNE Coloured by Split

In [ ]:
```
def make_thumb_b64(img_path, size=(64, 64)):
    try:
```

```
        img = Image.open(img_path).convert('RGB')
        img.thumbnail(size, Image.LANCZOS)
        buf = io.BytesIO()
        img.save(buf, format='PNG')
        return '' + base64.b64encode(buf.getvalue()
    except Exception:
        return ''


print('Building thumbnail cache (may take ~2 min)...')
thumbs = []
for _, row in tqdm(df.iterrows(), total=len(df), desc='Thumbnails'):
    p = DATASET_ROOT / row['split'] / 'images' / (row['img_name'] + '.jpg
    thumbs.append(make_thumb_b64(p))
df['thumb'] = thumbs
print('Done.')

SPLIT_COLORS = {'train': '#1f77b4', 'val': '#ff7f0e', 'test': '#2ca02c'}

fig5 = go.Figure()
for split in ['train', 'val', 'test']:
    mask = df['split'] == split
    sub  = df[mask]
    fig5.add_trace(go.Scatter(
        x=sub['tsne_0'], y=sub['tsne_1'],
        mode='markers',
        name=split,
        marker=dict(size=4, color=SPLIT_COLORS[split], opacity=0.7),
        customdata=sub[['img_name', 'sequence', 'thumb']].values,
        hovertemplate=(
            '<b>%{customdata[0]}</b><br>'
            'seq: %{customdata[1]}<br>'
            '<img src="%{customdata[2]}" width=64 height=64><extra></extr
        ),
    ))

fig5.update_layout(
    title='t-SNE of LaRS embeddings -- coloured by split',
    xaxis_title='t-SNE dim 0',
    yaxis_title='t-SNE dim 1',
    width=900, height=650,
    legend_title='Split',
)
fig5.show()
```

Section 6 - t-SNE Coloured by Sequence

In [ ]:

```
seq_counts = df['sequence'].value_counts()
major_seqs = seq_counts[seq_counts >= 5].index.tolist()
df['seq_label'] = df['sequence'].where(df['sequence'].isin(major_seqs), o
print('Major sequences (>=5 images):', len(major_seqs))
print('Images grouped as other:', (df['seq_label'] == 'other').sum())

import plotly.express as px
all_labels = sorted([s for s in df['seq_label'].unique() if s != 'other']
palette = px.colors.qualitative.Alphabet + px.colors.qualitative.Dark24 +
color_map = {label: palette[i % len(palette)] for i, label in enumerate(a
color_map['other'] = '#aaaaaa'
```

```
fig6 = go.Figure()
for label in all_labels:
    mask = df['seq_label'] == label
    sub  = df[mask]
    fig6.add_trace(go.Scatter(
        x=sub['tsne_0'], y=sub['tsne_1'],
        mode='markers',
        name=label,
        marker=dict(size=4, color=color_map[label], opacity=0.7),
        customdata=sub[['img_name', 'split', 'thumb']].values,
        hovertemplate=(
            '<b>%{customdata[0]}</b><br>'
            'split: %{customdata[1]}<br>'
            '<img src="%{customdata[2]}" width=64 height=64><extra></extr
        ),
    ))

fig6.update_layout(
    title='t-SNE of LaRS embeddings -- coloured by sequence',
    xaxis_title='t-SNE dim 0',
    yaxis_title='t-SNE dim 1',
    width=1200, height=700,
    legend_title='Sequence',
    showlegend=(len(all_labels) <= 60),
)
fig6.show()
```

Section 7 - Cosine Similarity Distribution (OOD Detection)

In [ ]:
```
from sklearn.preprocessing import normalize as sk_normalize

spatial_norm = sk_normalize(spatial_arr, norm='l2')

train_mask = (df['split'] == 'train').values
val_mask   = (df['split'] == 'val').values
test_mask  = (df['split'] == 'test').values

train_emb = spatial_norm[train_mask]
val_emb   = spatial_norm[val_mask]
test_emb  = spatial_norm[test_mask]

print('Computing max cosine similarity...')
print('val:', val_emb.shape[0], 'test:', test_emb.shape[0], 'vs train:',


def max_cosine_sim(query_emb, ref_emb, chunk=200):
    results = []
    for i in tqdm(range(0, len(query_emb), chunk), desc='Cosine sim'):
        batch = query_emb[i:i+chunk]
        sims  = batch @ ref_emb.T
        results.append(sims.max(axis=1))
    return np.concatenate(results)


max_sim_val  = max_cosine_sim(val_emb,  train_emb)
max_sim_test = max_cosine_sim(test_emb, train_emb)
```

```
df.loc[val_mask,  'max_train_sim'] = max_sim_val
df.loc[test_mask, 'max_train_sim'] = max_sim_test

fig7 = go.Figure()
fig7.add_trace(go.Histogram(x=max_sim_val,  name='val',  opacity=0.6, nbi
fig7.add_trace(go.Histogram(x=max_sim_test, name='test', opacity=0.6, nbi
fig7.add_vline(x=0.5, line_dash='dash', line_color='red',  annotation_tex
fig7.add_vline(x=0.7, line_dash='dash', line_color='gray', annotation_tex
fig7.update_layout(
    barmode='overlay',
    title='Max cosine similarity to training set',
    xaxis_title='Max cosine similarity',
    yaxis_title='Count',
    width=850, height=500,
)
fig7.show()

for split_name, sims in [('val', max_sim_val), ('test', max_sim_test)]:
    print('---', split_name, '---')
    print('  median max-sim :', round(float(np.median(sims)), 4))
    print('  mean max-sim   :', round(float(np.mean(sims)), 4))
    for thr in [0.5, 0.7]:
        pct = 100 * (sims < thr).mean()
        print('  % below', thr, '  :', round(float(pct), 1), '%')

test_df = df[test_mask].copy()
test_df['max_train_sim'] = max_sim_test
ood10 = test_df.nsmallest(10, 'max_train_sim')[['img_name', 'sequence', '
print('=== 10 most OOD test images (lowest max cosine similarity) ===')
print(ood10.to_string(index=False))
```

Section 8 - Report Generation

In [ ]:

```
import subprocess

NOTEBOOK_PATH = ANALYSIS_DIR / 'analysis.ipynb'
HTML_PATH     = ANALYSIS_DIR / 'report.html'

pdf_ok = False
if not REPORT_PATH.exists():
    print('Attempting nbconvert --to webpdf ...')
    r = subprocess.run(
        ['jupyter', 'nbconvert', '--to', 'webpdf',
         '--output', str(REPORT_PATH), str(NOTEBOOK_PATH)],
        capture_output=True, text=True
    )
    if r.returncode == 0 and REPORT_PATH.exists():
        pdf_ok = True
        print('PDF generated at', REPORT_PATH)
    else:
        print('webpdf failed:', r.stderr[-300:])

if not pdf_ok and not REPORT_PATH.exists():
    print('Converting notebook to HTML first ...')
    r = subprocess.run(
        ['jupyter', 'nbconvert', '--to', 'html',
         '--output', str(HTML_PATH), str(NOTEBOOK_PATH)],
```

```
        capture_output=True, text=True
    )
    if r.returncode == 0:
        print('HTML saved at', HTML_PATH)
        print('Converting HTML to PDF via weasyprint ...')
        try:
            from weasyprint import HTML as WP_HTML
            WP_HTML(filename=str(HTML_PATH)).write_pdf(str(REPORT_PATH))
            pdf_ok = True
            print('PDF generated at', REPORT_PATH)
        except Exception as e:
            print('weasyprint failed:', e)
    else:
        print('HTML export failed:', r.stderr[-300:])

if not pdf_ok:
    print('All PDF methods failed -- report available as HTML only.')
    print('HTML report:', HTML_PATH)
else:
    print('Report ready:', REPORT_PATH)
```

Section 9 - Summary Statistics

In [ ]:

```
from IPython.display import display, Markdown

rows_out = []
for split in ['train', 'val', 'test']:
    mask = df['split'] == split
    n_img  = int(mask.sum())
    n_seq  = int(df[mask]['sequence'].nunique())
    if 'max_train_sim' in df.columns:
        sim_vals = df.loc[mask & df['max_train_sim'].notna(), 'max_train_
        if len(sim_vals) > 0:
            mean_sim = float(sim_vals.mean())
            pct_ood  = float(100 * (sim_vals < 0.7).mean())
        else:
            mean_sim = pct_ood = float('nan')
    else:
        mean_sim = pct_ood = float('nan')
    rows_out.append({'split': split, 'n_images': n_img, 'n_sequences': n_
                     'mean_max_sim': mean_sim, 'pct_ood_07': pct_ood})

import math
def fmt(v):
    if isinstance(v, float) and math.isnan(v): return 'N/A'
    if isinstance(v, float): return f'{v:.3f}'
    return str(v)

md_lines = [
    '| split | n_images | n_sequences | mean_max_sim | % OOD (<0.7) |',
    '|-------|----------|-------------|--------------|--------------|',
]
for r in rows_out:
    md_lines.append(
        '| ' + r['split'] + ' | ' + fmt(r['n_images']) + ' | ' +
        fmt(r['n_sequences']) + ' | ' + fmt(r['mean_max_sim']) + ' | ' +
        fmt(r['pct_ood_07']) + '% |'
    )
```

```
md_table = '\n'.join(md_lines)
display(Markdown(md_table))
print(md_table)

print('Analysis complete.')
model_used = meta.get('model_name', 'unknown')
print('Model used:', model_used)
```